实验 4　openEuler 内核模块编程实践

一、实验目标

（1）了解 openEuler 内核基本结构。

（2）掌握 openEuler 内核编译步骤。

二、实验背景知识

2.1　openEuler 内核模块

openEuler 操作系统的内核是单一体系结构(monolithic kernel)的。也就是说，整个内核是一个单独的非常大的程序。与单一体系结构相对的是微内核体系结构(micro kernel)，比如 Windows NT 采用的就是微内核体系结构。微内核体系结构特点在于其只实现一些最基本的服务，如创建和删除进程、内存管理、中断管理等等。而文件系统、网络协议等其它部分都在微内核外的用户空间里运行。

这两种体系的内核各有优缺点。使用微内核的操作系统具有很好的可扩展性，但这样的操作系统由于不同层次之间的消息传递要花费一定的代价所以效率比较低。对单一体系结构的操作系统来说，所有的模块都集成在一起，系统的速度和性能都很好，但是可扩展性和维护性就相对比较差。

正是为了改善单一体系结构的缺点，Linux 操作系统使用了一种全新的内核模块机制。用户可以根据需要，在不需要对内核重新编译的情况下，模块能动态地装入内核或从内核移出。

内核模块是在内核空间运行的程序，实际上是一种目标对象文件，没有链接，不能独立运行，但是其代码可以在运行时链接到系统中作为内核的一部分运行或从内核中取下，从而可以动态扩充内核的功能。模块通常由一组函数和数据结构组成，用来实现一种文件系统，一个驱动程序，或其它内核上层的功能。模块机制的完整叫法应该是动态可加载内核模块(Loadable Kernel Module)，一般就简称为模块。与前面讲到的运行在微内核体系操作系统的外部用户空间的进程不同，模块不是作为一个进程执行的，而像其他静态连接的内核函数一样，它在内核态代表当前进程执行。使用模块的优点在于：

(1)使得内核更加紧凑和灵活

(2)修改内核时，不必全部重新编译整个内核，可节省不少时间，避免人工操作的错误。系统中如果需要使用新模块，只要编译相应的模块然后使用特定用户空间的程序将模块插入即

可。

(3)模块可以不依赖于某个固定的硬件平台。

(4)模块的目标代码一旦被链接到内核，它的作用和静态链接的内核目标代码完全等价。所以，当调用模块的函数时，无须显式的消息传递。

但是，内核模块的引入同时也带来一定的问题：

(1)由于内核所占用的内存是不会被换出的，所以链接进内核的模块会给整个系统带来一定的性能和内存利用方面的损失。

(2)装入内核的模块就成为内核的一部分，可以修改内核中的其他部分，因此，模块的使用不当会导致系统崩溃。

(3)为了让内核模块能访问所有内核资源，模块在装入和卸载模块时需要修改符号表。

(4)模块会使用到其它模块的功能，所以，内核要维护模块之间的依赖性。

模块是和内核在同样的地址空间运行的，模块编程在一定意义上说也就是内核编程。但是并不是内核中所有的地方都可以使用模块。模块一般在设备驱动程序、文件系统等地方使用，而对 Linux 内核中极为重要的地方，如进程管理和内存管理等，仍难以通过模块来实现，还是需要直接对内核进行修改。

在 Linux 内核源程序中，经常需要使用利用内核模块实现某些功能，比如文件系统、SCSI 高级驱动程序、CD-ROM 驱动程序以及以太网驱动程序等等。

2.2 tasklet

tasklet 是 openEuler 中断处理机制中的软中断延迟机制。引入 tasklet，最主要的是考虑支持 SMP（多处理，Symmetrical Multi-Processing），提高多个 CPU 的利用率。不同的 tasklet 可以在不同的 CPU 上运行。tasklet 可以理解为 softirq（软中断）的派生，它的调度时机和软中断一样。对于内核中需要延迟执行的多数任务都可以用 tasklet 来完成。由于 tasklet 本身已经进行了同步保护，所以使用 tasklet 比软中断要简单的多，tasklet 不需要考虑 SMP 下的并行问题，同时比 workqueues 有着更好的性能。通常 tasklet 作为中断下半部来使用。

tasklet 由 tasklet_struct 结构体来表示，每一个这样的结构体就表示一个 tasklet。在<linux/interrupt.h>中可以看到如下的定义：

```
struct tasklet_struct
{
        struct tasklet_struct *next;
        unsigned long state;
        atomic_t count;
```

```
    void (*func)(unsigned long);
    unsigned long data;
};
```

说明：

struct tasklet_struct *next;	代表链表中的下一个 tasklet
unsigned long state;	代表此刻 tasklet 的状态，一般为 TASKLET_STATE_SCHED，表示此 tasklet 已被调度且正准备运行；此变量还可取 TASKLET_STATE_RUN，表示正在运行，但只用在多处理器的情况下。
atomic_t count;	引用计数器，只有当其值为 0 时候，tasklet 才会被激活；否则被禁止，不能被执行。
void (*func)(unsigned long);	函数指针，它指向 tasklet 处理函数，这个处理函数的唯一参数为 data
unsigned long data;	上述处理函数的参数

在使用 tasklet 前，必须首先创建一个 tasklet_struct 类型的变量。通常有两种方法：静态创建和动态创建。

在<linux/interrupt.h>中的两个宏：

```
#define DECLARE_TASKLET(name,func,data) \
struct tasklet_struct name = { NULL, 0, ATOMIC_INIT(0), func, data }

#define DECLARE_TASKLET_DISABLED(name, func, data) \
struct tasklet_struct name = {NULL, 0 , ATOMIC_INIT(1), func, data }
```

通过第一个宏创建的 tasklet 处于激活状态，通过调度函数被挂起尽而被内核执行；通过第二个宏创建的 tasklet 处于禁止状态。

从两个宏的定义可以看到，所谓的静态创建或动态创建就是直接定义个一个名为 name 的 tasklet_struct 类型的变量，并将宏中各个参数相应的赋值给这个 name 变量的各个成员。

注意，两个宏在功能上差异就在于对 name 变量 count 成员的赋值上。具体原因见上述结构体说明。

tasklet 的结构体中包含处理函数的函数指针 func，它指向的是这样的一个函数：

```
void tasklet_handler(unsigned long data);
```

这个函数需要我们自己来实现。

创建好处理函数之后，我们还要通过如下的方法对 tasklet 进行初始化与调度：

```
void tasklet_init(struct tasklet_struct *t, void (*func)(unsigned long), unsigned long data);
/*  初始化 tasklet，func 指向要执行的函数，data 为传递给函数 func 的参数  */
tasklet_schedule(&my_tasklet)                    /*调度执行指定的 tasklet*/
```

```
void tasklet_kill(struct tasklet_struct *t)          /*移除指定 tasklet*/
void tasklet_disable(struct tasklet_struct *t)       /*禁用指定 tasklet*/
void tasklet_enable(struct tasklet_struct *t)        /*启用先前被禁用的 tasklet*/
```

2.3 openEuler 信号处理机制

openEuler 信号机制是一种进程间异步的通信机制，每个进程在运行时，都要通过信号机制来检查是否有信号到达，若有，便中断正在执行的程序，转向与该信号相对应的处理程序，以完成对该事件的处理；处理结束后再返回到原来的断点继续执行。实质上，信号机制是对中断机制的一种模拟，在实现上是一种软中断。

信号的生成来自内核，信号的请求来自 3 个地方：

(1)用户：用户能够通过终端按键产生信号，例如；

①ctrl+c ----> 2) SIGINT(终止、中断）

②ctrl+\ ----> 3)SIGQUIT(退出）

③ctrl+z ----> 20)SIGTSTP（暂时、停止）

④终端驱动程序分配给信号控制字符的其他任何键来请求内核产生信号

(2)内核：当进程执行出错时，内核会给进程发送一个信号，例如非法段存取(内存访问违规)、浮点数溢出等；

(3)进程：一个进程可以通过系统调用 kill 给另一个进程发送信号，一个进程可以通过信号和另外一个进程进行通信。

需要说明的是，信号只是用于通知进程发生了某个事件，除了信号本身的信息之外，并不具备传递用户数据的功能。

每个信号都有自己的响应动作，当接收到信号时，进程会根据信号的响应动作执行相应的操作，信号的响应动作有以下几种：

(1)中止进程(Term)

(2)忽略信号(Ign)

(3)中止进程并保存内存信息(Core)

(4)停止进程(Stop)

(5)继续运行进程(Cont)

用户可以通过 signal 或 sigaction 函数修改信号的响应动作（也就是常说的“注册信号”）。另外，在多线程中，各线程的信号响应动作都是相同的，不能对某个线程设置独立的响应动作。

Linux 支持的信号类型可以参考下面给出的列表。

信号	值	动作	说明
SIGHUP	1	Term	终端控制进程结束(终端连接断开)
SIGINT	2	Term	用户发送INTR字符(Ctrl+C)触发
SIGQUIT	3	Core	用户发送QUIT字符(Ctrl+/)触发
SIGILL	4	Core	非法指令(程序错误、试图执行数据段、栈溢出等)
SIGABRT	6	Core	调用abort函数触发
SIGFPE	8	Core	算术运行错误(浮点运算错误、除数为零等)
SIGKILL	9	Term	无条件结束程序(不能被捕获、阻塞或忽略)
SIGSEGV	11	Core	无效内存引用(试图访问不属于自己的内存空间、对只读内存空间进行写操作)
SIGPIPE	13	Term	消息管道损坏(FIFO/Socket通信时，管道未打开而进行写操作)
SIGALRM	14	Term	时钟定时信号
SIGTERM	15	Term	结束程序(可以被捕获、阻塞或忽略)
SIGUSR1	30,10,16	Term	用户保留
SIGUSR2	31,12,17	Term	用户保留
SIGCHLD	20,17,18	Ign	子进程结束(由父进程接收)
SIGCONT	19,18,25	Cont	继续执行已经停止的进程(不能被阻塞)
SIGSTOP	17,19,23	Stop	停止进程(不能被捕获、阻塞或忽略)
SIGTSTP	18,20,24	Stop	停止进程(可以被捕获、阻塞或忽略)
SIGTTIN	21,21,26	Stop	后台程序从终端中读取数据时触发
SIGTTOU	22,22,27	Stop	后台程序向终端中写数据时触发

图 4.1　Linux 支持的信号类型

信号是异步的，这就涉及信号何时接收、何时处理的问题。我们知道，函数运行在用户态，当遇到系统调用、中断或是异常的情况时，程序会进入内核态。信号涉及到了这两种状态之间的转换，过程可以先看一下下面的示意图：

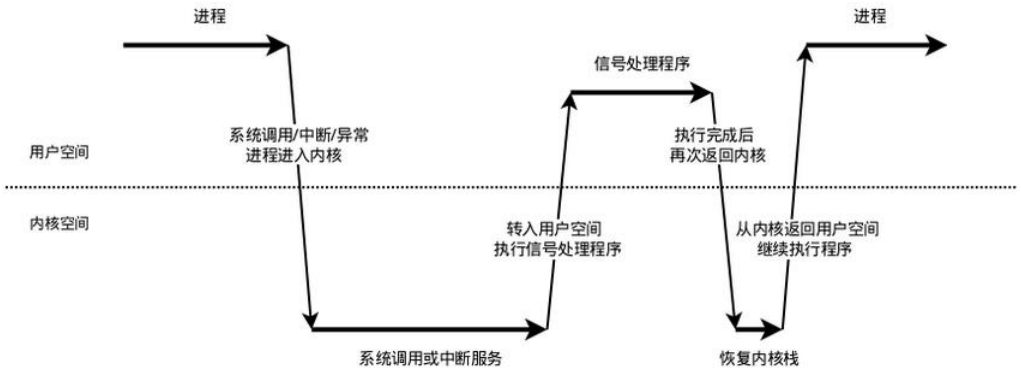


图 4.2　信号机制

(1)信号的接收

接收信号的任务是由内核负责的，当内核接收到信号后，会将其放到对应进程的信号队列中，同时向进程发送一个中断，使其陷入内核态。注意，此时信号还只是在队列中，对进程来说暂时是不知道有信号到来的。

(2)信号的检测

进程陷入内核态后，有两种场景会对信号进行检测：

①进程从内核态返回到用户态前进行信号检测；

②进程在内核态中，从睡眠状态被唤醒的时候进行信号检测；

当发现有新信号时，便会进入下一步，信号的处理。

(3)信号的处理

信号处理函数是运行在用户态的，调用处理函数前，内核会将当前内核栈的内容备份拷贝到用户栈上，并且修改指令寄存器（eip）将其指向信号处理函数。

接下来进程返回到用户态中，执行相应的信号处理函数。

信号处理函数执行完成后，还需要返回内核态，检查是否还有其它信号未处理。如果所有信号都处理完成，就会将内核栈恢复（从用户栈的备份拷贝回来），同时恢复指令寄存器（eip）将其指向中断前的运行位置，最后回到用户态继续执行进程。

至此，一个完整的信号处理流程便结束了。

2.4 内核时钟以及定时器

(1)时钟

时钟是操作系统中用来管理系统时间的组件。时钟通常由内核提供，可以使用时钟相关的头文件和函数来进行操作和管理。

头文件：<time.h>

常用函数：

①time(): 获取当前系统时间的时间戳，返回值为 time_t 类型，单位为秒。

②clock(): 获取当前进程已经消耗的 CPU 时间，返回值为 clock_t 类型，单位为时钟周期。

③clock_gettime(): 获取系统时钟的时间，可以精确到纳秒级别。

(2)定时器

定时器（有时也称为动态定时器或内核定时器）是管理内核流逝的时间的基础。内核经常需要推后执行某些代码，我们需要的是一种工具，能够使工作在指定时间点上执行——不长不短，正好在希望的时间点上。内核定时器正是解决这个问题的理想工具。

定时器的使用很简单。你只需要执行一些初始化工作，设置一个超时时间，指定超时发生后执行的函数，然后激活定时器就可以了。指定的函数在定时器到期时自动执行。注意定时器并不周期运行，它在超时后就自行撤销，这也正是这种定时器被称为动态定时器的一个原因：动态定时器不断地创建和撤销，而且它的运行次数也不受限制。

头文件：<sys/time.h>

常用函数：

①alarm(): 设置定时器，在指定的时间间隔之后发送 SIGALRM 信号，然后触发处理程序。

②setitimer(): 设置定时器，可以实现周期性触发，也可以只触发一次。

③gettimeofday(): 获取当前时间和日期，可以精确到微秒级别。

三、实验详细过程

3.1 案例 1

编写内核模块，使用 tasklet 实现打印 helloworld。加载、卸载模块并查看模块打印信息。

(1)首先创建一个新的文件夹并且进入到这个文件夹 ：

```
[root@localhost ~]# mkdir task1
[root@localhost task1]# cd task1
```

(2)进入文件夹后首先创建 tasklet_intertupt.c 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task1]# touch tasklet_intertupt.c
[root@localhost task1]# vi tasklet_intertupt.c
```

(3)按下 i 键进入 INSERT 模式并输入以下代码：

```
#include <linux/module.h>
#include <linux/interrupt.h>
MODULE_LICENSE("GPL");
static struct tasklet_struct my_tasklet;
static void tasklet_handler(unsigned long data)
{
    printk("Hello World! tasklet is working...\n");
}
static int __init mytasklet_init(void)
{
    printk("Start tasklet module...\n");
    tasklet_init(&my_tasklet, tasklet_handler, 0);
    tasklet_schedule(&my_tasklet);
    return 0;
}
static void __exit mytasklet_exit(void)
{
```

```
    tasklet_kill(&my_tasklet);
    printk("Exit tasklet module...\n");
}
module_init(mytasklet_init);
module_exit(mytasklet_exit);
```

(4)输入完成后按 esc 键输入 “:wq”保存并退出编辑 tasklet_intertupt.c 文件。

(5)之后编辑 Makefile 文件,创建 Make 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task1]# touch Makefile
[root@localhost task1]# vi Makefile
```

(6)按下 i 键进入 INSERT 模式并输入以下代码：

```
ifneq ($(KERNELRELEASE),)
    obj-m := tasklet_intertupt.o
else
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

请注意，KERNELDIR 后面的文件夹一定要是第三章中编译内核的文件夹目录，obj-m 后面的.o 文件的名称必须与上面创建的.c 文件名称相同。

(7)输入完成后按 esc 键输入 “:wq”保存并退出编辑 Makefile 文件。

(8)两个文件都创建完成后在 task1 文件夹中输入命令 make 开始编译：

```
[root@localhost task1]# make
```

(9)编译结束后输入命令 ls 可以查看当前文件夹下的文件：

```
[root@localhost task1]# ls
Makefile modules.order Module.symvers tasklet_intertupt.c tasklet_intertupt.ko tasklet_intertupt.mod.c tasklet_intertupt.mod.o  tasklet_intertupt.o
```

(10)在命令中输入命令 insmod tasklet_intertupt.ko 插入模块：

```
[root@localhost task1]# insmod tasklet_intertupt.ko
```

(11)如果没有额外信息则代表模块插入成功，在命令行中输入命令 dmesg | tail -n 2 查看模块打印信息：

```
[root@localhost task1]# demsg | tail -n 2
[    25.280985] Start tasklet module...
[    25.280995] Hello World! tasklet is working...
```

(12)输出上述信息则代表内核模块成功运行，接下来在命令行中输入 rmmod tasklet_intertupt.ko 以卸载模块：

```
[root@localhost task1]# rmmod tasklet_intertupt.ko
```

3.2 案例 2

在用户态编写一个信号捕获程序，捕获终端按键信号（包括 ctrl+c、ctrl+z、ctrl+\）。编译上述程序后运行，在终端输入按键信号（ctrl+c、ctrl+z、ctrl+\），查看输出信息。

(1)首先创建一个新的文件夹并且进入到这个文件夹：

```
[root@localhost ~]# mkdir task2
[root@localhost task3]# cd task2
```

(2)进入文件夹后首先创建 catch_signal.c 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task2]# touch catch_signal.c
[root@localhost task2]# vi catch_signal.c
```

(3)按下 i 键进入 INSERT 模式并输入以下代码：

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

void signal_handler(int sig)
{
    switch(sig){
        case SIGINT:
            printf("\nGet a signal:SIGINT. You pressed ctrl+c.\n");
            break;
        case SIGQUIT:
            printf("\nGet a signal:SIGQUIT. You pressed ctrl+\\.\n");
            break;
        case SIGTSTP:
            printf("\nGet a signal:SIGHUP. You pressed ctrl+z.\n");
            break;
    }
    exit(0);
}
int main()
{
    printf("Current process ID is %d\n", getpid());
    signal(SIGINT, signal_handler);
    signal(SIGQUIT, signal_handler);
    signal(SIGTSTP, signal_handler);
```

```
    for(;;);
}
```

(4)在命令中输入命令 gcc catch_signal.c -o catch_signal：

```
[root@localhost task2]# gcc catch_signal.c -o catch_signal
```

(5)按下 ctrl+c 键查看输出：

```
[root@localhost task2]# ./ catch_signal
Current process ID is 224881
^C
Get a signal:SIGINT. You pressed ctrl+c.
```

(6)输出上述内容代表信号接受成功。

3.3 案例 3

编写内核模块，调用内核时钟接口，打印出系统当前时间。格式示例：2023-04-20 19:54:31，加载、卸载模块并查看模块打印信息。

(1)首先创建一个新的文件夹并且进入到这个文件夹：

```
[root@localhost ~]# mkdir task3
[root@localhost task3]# cd task3
```

(2)进入文件夹后首先创建 current_time.c 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task3]# touch current_time.c
[root@localhost task3]# vi current_time.c
```

(3)按下 i 键进入 INSERT 模式并输入以下代码：

```
#include <linux/module.h>
#include <linux/time.h>
#include <linux/rtc.h>

MODULE_LICENSE("GPL");

struct timeval tv;
struct rtc_time tm;

static int __init currenttime_init(void)
{
    int year, mon, day, hour, min, sec;
    printk("Start current_time module...\n");
    do_gettimeofday(&tv);
    rtc_time_to_tm(tv.tv_sec, &tm);
    year = tm.tm_year + 1900;
    mon = tm.tm_mon + 1;
    day = tm.tm_mday;
    hour = tm.tm_hour + 8;
```

```
    min = tm.tm_min;
    sec = tm.tm_sec;
    printk("Current time: %d-%02d-%02d %02d:%02d:%02d\n", year, mon, day, hour, min, sec);
    return 0;
}

static void __exit currenttime_exit(void)
{
    printk("Exit current_time module...\n");
}
module_init(currenttime_init);
module_exit(currenttime_exit);
```

(4)输入完成后按 esc 键输入“:wq”保存并退出编辑 current_time.c 文件。

(5)之后编辑 Makefile 文件=，创建 Make 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task3]# touch Makefile
[root@localhost task3]# vi Makefile
```

(6)按下 i 键进入 INSERT 模式并输入以下代码：

```
ifneq ($(KERNELRELEASE),)
    obj-m := current_time.o
else
    KERNELDIR ?= /root/ kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

请注意，KERNELDIR 后面的文件夹一定要是第三章中编译内核的文件夹目录，obj-m 后面的.o 文件的名称必须与上面创建的.c 文件名称相同。

(7)输入完成后按 esc 键输入“:wq”保存并退出编辑 Makefile 文件。

(8)两个文件都创建完成后在 task3 文件夹中输入命令 make 开始编译：

```
[root@localhost task3]# make
```

(9)编译结束后输入命令 ls 可以查看当前文件夹下的文件：

```
[root@localhost task3]# ls
current_time.c  current_time.ko  current_time.mod.c  current_time.mod.o  current_time.o  Makefile  modules.order  Module.symvers
```

(10)在命令中输入命令 insmod current_time.ko 插入模块：

```
[root@localhost task3]# insmod current_time.ko
```

(11)如果没有额外信息则代表模块插入成功，在命令行中输入命令 dmesg | tail -n 2 查看模块打印信息：

```
[root@localhost task3]# demsg | tail -n 2
[      26.688667] Start current_time module...
[      25.688668] Current time:2022-08-04 19:49:58
```

(12)输出上述信息则代表内核模块成功运行，接下来在命令行中输入 rmmod current_time.ko 以卸载模块：

```
[root@localhost task3]# rmmod current_time.ko
```

3.4 案例 4

编写内核模块程序，实现一个 timer，该定时器延时 10 秒后打印“hello,world”。加载、卸载模块并查看模块打印信息。

(1)首先创建一个新的文件夹并且进入到这个文件夹：

```
[root@localhost ~]# mkdir task4
[root@localhost task4]# cd task4
```

(2)进入文件夹后首先创建 timer_example.c 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task4]# touch timer_example.c
[root@localhost task4]# vi timer_example.c
```

(3)按下 i 键进入 INSERT 模式并输入以下代码：

```
#include <linux/module.h>
#include <linux/timer.h>

MODULE_LICENSE("GPL");

struct timer_list timer;

void print(struct timer_list *timer)
{
    printk("hello,world!\n");
}
static int __init timer_init(void)
{
    printk("Start timer_example module...\n");
    timer.expires = jiffies + 10 * HZ;
    timer.function = print;
    add_timer(&timer);
    return 0;
}
```

```
static void __exit timer_exit(void)
{
    printk("Exit timer_example module...\n");
}

module_init(timer_init);
module_exit(timer_exit);
```

(4)输入完成后按 esc 键输入“:wq”保存并退出编辑 timer_example.c 文件。

(5)之后编辑 Makefile 文件，创建 Make 文件并使用 vi 命令对其进行编辑：

```
[root@localhost task4]# touch Makefile
[root@localhost task4]# vi Makefile
```

(6)按下 i 键进入 INSERT 模式并输入以下代码：

```
ifneq ($(KERNELRELEASE),)
    obj-m := timer_example.o
else
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

注意，KERNELDIR 后面的文件夹一定要是第三章中编译内核的文件夹目录，obj-m 后面的.o 文件的名称必须与上面创建的.c 文件名称相同。

(7)输入完成后按 esc 键输入“:wq”保存并退出编辑 Makefile 文件。

(8)两个文件都创建完成后在 task4 文件夹中输入命令 make 开始编译：

```
[root@localhost task4]# make
```

(9)编译结束后输入命令 ls 可以查看当前文件夹下的文件：

```
[root@localhost task4]# ls
Makefile modules.order Module.symvers timer_example.c timer_example.ko timer_example.mod.c timer_example.mod.o timer_example.o
```

(10)在命令中输入命令 insmod timer_example.ko 插入模块：

```
[root@localhost task3]# insmod timer_example.ko
```

(11)如果没有额外信息则代表模块插入成功，在命令行中输入命令 dmesg -T | tail -n 2 查看模块打印信息：

```
[root@localhost task3]# demsg -T | tail -n 2
```

```
[Thu Apr    20 20:04:37 2022] Start timer_example module...
[Thu Apr    20 20:04:47 2023] hello,world!
```

(12)输出上述信息则代表内核模块成功运行，接下来在命令行中输入 rmmod timer_example.ko 以卸载模块：

```
[root@localhost task4]# rmmod timer_example.ko
```

四、实验讨论

(1)案例一中，我们使用了 tasklet 来实现打印"Hello World"。Tasklet 与其他内核机制（如软中断或工作队列）相比有什么优势和劣势？

(2)信号是在用户空间和内核空间之间进行通信的一种机制。在第二个案例中，我们编写了一个信号捕获程序。那么，用户态和内核态之间的信号传递是如何实现的？

(3)在第三个案例中，我们调用了内核时钟接口来获取系统当前时间。在 Linux 内核中，有哪些不同的时钟类型和时钟源？如何选择时钟类型和时钟源的？

(4)定时器是操作系统中经常使用的一个概念，可以在指定的时间间隔内执行某个操作。在第四个案例中，我们编写了一个内核模块来实现一个定时器。那么，如何确保该定时器能够准确地计时并执行相应的操作？